Bash Script -2

1. Create a bash script to check if a directory is available or not.

Step 1: -Create a bash file with Touch or vi command

Step 2: -Use shebang Character and shell path to be used for kernel

Step 3:-Take user input with read and with if condition search the folder. If the file is available print ‘Directory exits’ else ‘Directory does not exits’

Step 4:- Finally use fi command to escape the if condition

Step 5:- Now to run this file we either use chmod to modify its permission so that every one can read and execute this file (Or) Use bash command before the file path to run the bash file.

```
[root@ip-172-31-35-122 scripts]# vi checkdir.bash
[root@ip-172-31-35-122 scripts]# cat checkdir.bash
#!/bin/bash

echo 'Enter the directory path'

read dir

if [ -e $dir ];
        then
        echo 'Directory exists'
else
        echo "Directory doesn't exist"
fi
[root@ip-172-31-35-122 scripts]#
```

```
[root@ip-172-31-35-122 scripts]# bash checkdir.bash
Enter the directory path
/etc/passwd
Directory exists
[root@ip-172-31-35-122 scripts]#
```

2. Create a bash script to create multiple files.

Step 1: -Create a bash file with Touch or vi command

Step 2: -Use shebang Character and shell path to be used for kernel

Step 3:-Taking three input first one for folder path , second one for number of files we want to create and last one is for file name

Step 4:- First we take folder path we want to save the file. Check whether the folder exists or not

Step 5:- Then take numerical input to create a for loop for creating file

Step 6:-In for loop we take a input to create a file with Touch command

Step 7:- To end the if condition we use fi keyword and to end for loop we use done keyword

Step 8:- This way we can create many file at custom folder paths.

Step 9:- Finaly run this file with bash command

```
root@ip-172-31-35-122:/scripts
[root@ip-172-31-35-122 scripts]# vi createfile.bash
[root@ip-172-31-35-122 scripts]# cat createfile.bash
#!/bin/bash

echo 'Enter the path '
echo
read dir
echo

if [ -e "$dir" ];
then
        cd $dir
        echo 'Enter the number of file you want to create..'
        read n
        for (( i=1; i<=n; i++ ))
        do
                echo "Enter $i file name with extension: "
                read name
                if [ -e "$name" ];
                then
                        echo "$name :file already exists"
                else
                        touch $name
                        echo "$name file has been created in $dir"
                fi
        done
else
        echo 'Directory does not exist'

fi
[root@ip-172-31-35-122 scripts]# bash ./createfile.bash
Enter the path

/opt

Enter the number of file you want to create..
3
Enter 1 file name with extension:
tom.txt
tom.txt file has been created in /opt
Enter 2 file name with extension:
nginx.txt
nginx.txt file has been created in /opt
Enter 3 file name with extension:
dev.txt
dev.txt file has been created in /opt
[root@ip-172-31-35-122 scripts]#
```

```
root@ip-172-31-35-122:/opt
[root@ip-172-31-35-122 scripts]# cd /opt
[root@ip-172-31-35-122 opt]# ls
aws  dev.txt  nginx.txt  tom.txt
[root@ip-172-31-35-122 opt]#
```

3. Create a bash script to take a backup of a directory.

Step 1: -Create a bash file with Touch or vi command

Step 2: -Use shebang Character and shell path to be used for kernel

Step 3:- Here we store path to store this backup

Step 4:- Then we take input of directory which we want to create a backup .

Step 5:- Checking if the input folder path exists or not with -e (To check for files and folder in If condition)

Step 6:-If directory exists we create a compressed tar file with tar czvf command and print any message

Step 7:- if directory doesn't exist we send directory not found message.

Step 8:- Finally running this file with bash command

```
[root@ip-172-31-35-122 scripts]# vi backup.bash
[root@ip-172-31-35-122 scripts]# cat backup.bash
#!/bin/bash

echo 'Enter the folder for backup: '
echo
read dir
echo

tmp='/tmp/backup'

if [ -e $dir ];
then
        echo 'Enter backup name: '
        echo
        read back
        echo

        tar czvf $tmp/$back.tar.gz $dir
        echo 'Backup file is created!'
else
echo 'Directory Not found'
echo
echo 'Program Aborted!'
fi
```

```
[root@ip-172-31-35-122 scripts]# bash backup.bash
Enter the folder for backup:

/scripts

Enter backup name:

bash_scripts

tar: Removing leading `/' from member names
/scripts/
/scripts/basic.bash
/scripts/a
/scripts/b
/scripts/c
/scripts/variable.bash
/scripts/input.bash
/scripts/int-out.bash
/scripts/if-else.bash
/scripts/filepath.bash
/scripts/case.bash
/scripts/for.bash
/scripts/whilee.bash
/scripts/access.bash
/scripts/admin.bash
/scripts/checkdir.bash
/scripts/createfile.bash
/scripts/backup.bash
Backup file is created!
[root@ip-172-31-35-122 scripts]# cd /tmp
[root@ip-172-31-35-122 tmp]# ls
backup
systemd-private-a2bec9391f544b2b95a6fada63a4a4f0-chronyd.service-aqM3LO
systemd-private-a2bec9391f544b2b95a6fada63a4a4f0-dbus-broker.service-QYwU1m
[root@ip-172-31-35-122 tmp]# cd backup
[root@ip-172-31-35-122 backup]# ls
bash_scripts.tar.gz
[root@ip-172-31-35-122 backup]#
```

4. Create a bash script to install Nginx on an EC2 server.

Step 1: -Create a bash file with Touch or vi command

Step 2: -Use shebang Character and shell path to be used for kernel

Step 3:- First update all the packages with yum update

Step 4:- Then checking if nginx is already installed or not

Step 5:- If installed then print a message as Already exist

Step 6:- If not installed then install the package and print the message.

Step 7:- Finally run this file with help of bash command

root@ip-172-31-35-122:/scripts

```
[root@ip-172-31-35-122 scripts]# vi nginxinst.bash
[root@ip-172-31-35-122 scripts]# cat nginxinst.bash
#!/bin/bash

echo
echo 'Updating Packages'
sudo yum update
echo
echo  'Checking Package Info'
echo

if rpm -q nginx;then
        echo 'Package is already installed'
else
        echo 'Installing the Nginx Package'
        sudo yum install nginx -y
        echo
        echo 'Installed the Nginx package'
fi
```

```
[root@ip-172-31-35-122 scripts]# bash ./nginxinst.bash

Updating Packages
Last metadata expiration check: 0:02:32 ago on Tue Oct 20 05:24:51 2026.
================================================================================
WARNING:
  A newer release of "Amazon Linux" is available.

  Available Versions:

  Version 2023.12.20261016:
    Run the following command to upgrade to 2023.12.20261016:

      dnf upgrade --releasever=2023.12.20261016

    Release notes:
     https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.12.20261016.html

================================================================================
Dependencies resolved.
Nothing to do.
Complete!

Checking Package Info

package nginx is not installed
Installing the Nginx Package
Last metadata expiration check: 0:02:33 ago on Tue Oct 20 05:24:51 2026.
Dependencies resolved.
================================================================================
 Package                                        Architecture
================================================================================
Installing:
 nginx                                          x86_64
Installing dependencies:
 generic-logos-httpd                            noarch
 gperftools-libs                                x86_64
 libunwind                                      x86_64
 nginx-core                                     x86_64
 nginx-filesystem                               noarch
 nginx-mimetypes                                noarch

Transaction Summary
================================================================================
Install  7 Packages
```

```
Installed:
  generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch       gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64
  nginx-core-1:1.30.5-1.amzn2023.0.1.x86_64               nginx-filesystem-1:1.30.5-1.amzn2023.0.1.noarch

Complete!

Installed the Nginx package
[root@ip-172-31-35-122 scripts]#
```

5. Create a bash script to install Apache Tomact on an EC2 server.

Step 1: -Create a bash file with Touch or vi command

Step 2: -Use shebang Character and shell path to be used for kernel

Step 3:- First update all the packages with yum update

Step 4:- Then checking if tomcat is already installed or not

Step 5:- If installed then print a message as Already exist

Step 6:- If not installed then install the package and print the message.

Step 7:- Finally run this file with help of bash command

```
[root@ip-172-31-35-122 scripts]# vi tomcatinst.bash
[root@ip-172-31-35-122 scripts]# cat tomcatinst.bash
#!/bin/bash

echo
echo 'Updating Packages'
sudo yum update
echo
echo  'Checking Package Info'
echo

if rpm -q tomcat10;then
        echo 'Package is already installed'
else
        echo 'Installing the Apache Tomcat Package'
        sudo yum install tomcat10 -y
        echo
        echo 'Installed the Apache Tomcat package'
fi
```

```
[root@ip-172-31-35-122 scripts]# bash ./tomcatinst.bash

Updating Packages
Last metadata expiration check: 0:06:47 ago on Tue Oct 20 05:30:59 2026.
Dependencies resolved.
Nothing to do.
Complete!

Checking Package Info

package tomcat is not installed
Installing the Apache Tomcat Package
Last metadata expiration check: 0:06:48 ago on Tue Oct 20 05:30:59 2026.
Dependencies resolved.
================================================================================
 Package                                 Architecture                 Version
================================================================================
Installing:
 tomcat10                                noarch                       1:10.1.60-1.amzn2023.0.1
Installing dependencies:
 apr                                     x86_64                       1.7.5-1.amzn2023.0.4
 cairo                                   x86_64                       1.18.0-4.amzn2023.0.3
 ecj                                     noarch                       1:4.23-1.amzn2023.0.4
 fonts-filesystem                        noarch                       1:2.0.5-12.amzn2023.0.2
 google-noto-fonts-common                noarch                       20240401-1.amzn2023.0.2
 google-noto-sans-vf-fonts               noarch                       20240401-1.amzn2023.0.2
 graphite2                               x86_64                       1.3.14-7.amzn2023.0.3
 harfbuzz                                x86_64                       7.0.0-2.amzn2023.0.2
 java-17-amazon-corretto-headless        x86_64                       1:17.0.20+13-1.amzn2023.1
 javapackages-filesystem                 noarch                       6.0.0-7.amzn2023.0.6
 javapackages-tools                      noarch                       6.0.0-7.amzn2023.0.6
 langpacks-core-font-en                  noarch                       3.0-21.amzn2023.0.4
 libX11                                  x86_64                       1.8.10-2.amzn2023.0.2
 libX11-common                           noarch                       1.8.10-2.amzn2023.0.2
 libXau                                  x86_64                       1.0.11-6.amzn2023.0.1
 libXext                                 x86_64                       1.3.6-1.amzn2023.0.1
 libXrender                              x86_64                       0.9.11-6.amzn2023.0.2
 libbrotli                               x86_64                       1.1.0-1.amzn2023.0.1
 libjpeg-turbo                           x86_64                       2.1.4-2.amzn2023.0.5
 libpng                                  x86_64                       2:1.6.37-10.amzn2023.0.13
 libxcb                                  x86_64                       1.17.0-1.amzn2023.0.1
 pixman                                  x86_64                       0.43.4-1.amzn2023.0.4
```

```
Installed:
  alsa-lib-1.2.7.2-1.amzn2023.0.4.x86_64                apr-1.7.5-1.amzn2023.0.4.x86_64
  dejavu-sans-fonts-2.37-16.amzn2023.0.2.noarch         dejavu-sans-mono-fonts-2.37-16.amzn2023.0.2.noarch
  ecj-1:4.23-1.amzn2023.0.4.noarch                      fontconfig-2.13.94-2.amzn2023.0.2.x86_64
  freetype-2.13.2-5.amzn2023.0.2.x86_64                 google-noto-fonts-common-20240401-1.amzn2023.0.2.noarch
  graphite2-1.3.14-7.amzn2023.0.3.x86_64                harfbuzz-7.0.0-2.amzn2023.0.2.x86_64
  javapackages-filesystem-6.0.0-7.amzn2023.0.6.noarch   javapackages-tools-6.0.0-7.amzn2023.0.6.noarch
  libX11-1.8.10-2.amzn2023.0.2.x86_64                   libX11-common-1.8.10-2.amzn2023.0.2.noarch
  libXext-1.3.6-1.amzn2023.0.1.x86_64                   libXrender-0.9.11-6.amzn2023.0.2.x86_64
  libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64             libpng-2:1.6.37-10.amzn2023.0.13.x86_64
  pixman-0.43.4-1.amzn2023.0.4.x86_64                   tomcat-native-2.0.16-1.amzn2023.0.1.x86_64
  tomcat10-el-5.0-api-1:10.1.60-1.amzn2023.0.1.noarch   tomcat10-jsp-3.1-api-1:10.1.60-1.amzn2023.0.1.noarch
  tomcat10-servlet-6.0-api-1:10.1.60-1.amzn2023.0.1.noarch  xml-common-0.6.3-56.amzn2023.0.2.noarch

Complete!

Installed the Apache Tomcat package
[root@ip-172-31-35-122 scripts]#
```

```
[root@ip-172-31-35-122 scripts]# bash ./tomcatinst.bash

Updating Packages
Last metadata expiration check: 0:09:00 ago on Tue Oct 20 05:30:59 2026.
Dependencies resolved.
Nothing to do.
Complete!

Checking Package Info

tomcat10-10.1.60-1.amzn2023.0.1.noarch
Package is already installed
[root@ip-172-31-35-122 scripts]#
```

6. Create a bash script to check if the nginx service is running , if not running then script should start the service.

Step 1: -Create a bash file with Touch or vi command

Step 2: -Use shebang Character and shell path to be used for kernel

Step 3:- First check its status with systemctl and use grep to find if it is active or not without printing anything (-q don't print anything)

Step 4:- If running print a message and end the if condition

Step 5:- If not start the package with systemctl and recheck the package status with similar if condition .

Step 6:- Then print its status with message.

Step 7:- Finally run this file with help of bash command

```
[root@ip-172-31-35-122 scripts]# vi run-nginx.bash
[root@ip-172-31-35-122 scripts]# cat run-nginx.bash
#!/bin/bash

echo
echo 'Checking the Package'
echo
if systemctl status nginx | grep -q 'active (running)'; then
        echo 'Package is already running and active'
else
        echo 'Package is inactive'
        echo
        echo 'starting the Package'
        sudo systemctl start nginx
        echo
        if systemctl status nginx | grep -q 'active (running)'; then
                echo 'Package started successfully'
        else
                echo 'Error in Starting the package'
        fi
fi
```

```
[root@ip-172-31-35-122 scripts]# bash ./run-nginx.bash

Checking the Package

Package is inactive

starting the Package

Package started successfully
[root@ip-172-31-35-122 scripts]#
```

7. Create a bash script for calculator.

Step 1: -Create a bash file with Touch or vi command

Step 2: -Use shebang Character and shell path to be used for kernel

Step 3: -Take two number on which you want to perform operations

Step 4:- Print some option and take user input on which operation to perform

Step 4:- Then use case and user choice to filter the operation and store the output from that to a variable to print

Step 5:- Also every case option should end with double single colon to work(;;).

Step 6:- After getting the output to quit the case command we use Esac.

Step 7:- Finally run this file with help of bash command

```
[root@ip-172-31-35-122 scripts]# vi calculator.bash
[root@ip-172-31-35-122 scripts]# cat calculator.bash
#!/bin/bash

echo "===== SIMPLE CALCULATOR ====="

echo 'Enter First number: '
read  num1
echo
echo 'Enter second number: '
read  num2

echo "Choose an operation:"
echo "1. Addition (+)"
echo "2. Subtraction (-)"
echo "3. Multiplication (*)"
echo "4. Division (/)"

read choice

case $choice in
    1)
        result=$((num1 + num2))
        echo "Result: $result"
        ;;
    2)
        result=$((num1 - num2))
        echo "Result: $result"
        ;;
    3)
        result=$((num1 * num2))
        echo "Result: $result"
        ;;
```

```
    4)
        if [ "$num2" -eq 0 ]; then
            echo "Error: Division by zero is not allowed."
        else
            result=$((num1 / num2))
            echo "Result: $result"
        fi
        ;;
    *)
        echo "Invalid choice!"
        ;;
esac
```

```
[root@ip-172-31-35-122 scripts]# bash ./calculator.bash
===== SIMPLE CALCULATOR =====
Enter First number:
10

Enter second number:
2
Choose an operation:
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
4
Result: 5
[root@ip-172-31-35-122 scripts]#
```

8. Create a bash script to check if a directory exists, if not then create a directory.

Step 1: -Create a bash file with Touch or vi command

Step 2: -Use shebang Character and shell path to be used for kernel

Step 3:- First taking the user input and checking whether folder exists or not with -f (for folder only in IF condition)

Step 4:- If the folder exist it will print a message as exists

Step 5:- If it is not found create a new directory with same name taken from user input

Step 6:- And print a message as folder created successfully

Step 7:- Finally run this file with help of bash command

```
[root@ip-172-31-35-122 scripts]# vi find-dir.bash
[root@ip-172-31-35-122 scripts]# cat find-dir.bash
#!/bin/bash

echo 'Enter the Directory: '
read dir

echo
echo 'Checking for Directory...'
echo
```

```
if [ -d $dir ]; then
        echo 'Directory Exists'
else
        echo 'There is no such Directory'
        echo
        echo 'Creating the directory'
        mkdir $dir
        echo 'Directory is succesfully created'
fi
```

```
[root@ip-172-31-35-122 scripts]# bash ./find-dir.bash
Enter the Directory:
/backup

Checking for Directory...

There is no such Directory

Creating the directory
Directory is succesfully created
```

9. Create a bash script to delete the last 3 lines of a file.

Step 1: -Create a bash file with Touch or vi command

Step 2: -Use shebang Character and shell path to be used for kernel

Step 3:- First taking user input of file name

Step 4:- Then checking if exist or not. If exist using Sed command deleting last line with $ to get last lines and d to delete it

Step 5:- Also using -i to permanently save the changes. Then printing the message as lines deleted

Step 6:- If does not exist then directly print a message.

Step 7:- Finally run this file with help of bash command

```
[root@ip-172-31-35-122 scripts]# vi delete-line.bash
[root@ip-172-31-35-122 scripts]# cat delete-line.bash
#!/bin/bash

echo "Enter the file name:"
read filename

if [ -f $filename ]; then
    sed -i '$d' $filename
    sed -i '$d' $filename
    sed -i '$d' $filename
    echo "Last 3 lines deleted successfully."
else
    echo "File does not exist."
fi
[root@ip-172-31-35-122 scripts]#
```

```
[root@ip-172-31-35-122 scripts]# cat text.txt
Line 1
Line 2
Line 3
Line 4
Line 5
Line 6
Line 7
Line 8
Line 9
[root@ip-172-31-35-122 scripts]# bash ./delete-line.bash
Enter the file name:
text.txt
Last 3 lines deleted successfully.
[root@ip-172-31-35-122 scripts]#
```

```
[root@ip-172-31-35-122 scripts]# cat text.txt
Line 1
Line 2
Line 3
Line 4
Line 5
Line 6
[root@ip-172-31-35-122 scripts]#
```

10. Create a bash script to monitor CPU and if it is ,more than 80% then send email notification.

Step 1: -Create a bash file with Touch or vi command

Step 2: -Use shebang Character and shell path to be used for kernel

Step 3:- First we get the snap of CPU usage with top and -bn1 tag. B tag is to get the usage in plain text format, n1 tag to refresh one time before closing.

Step 4:- Then get CPU(s) column from this. Use awk to separate by column and print fourth column. Finally subtracting the output from 100%.

Step 5:- Store this in variable and then remove the decimal with %.*

Step 6:- Then check if the stored value is greater than 80 or not

Step 7:-If greater then use mail command to send a report

Step 8:- If not then print a normal message with the utilization value.

Step 9:- Finally run this file with help of bash command

```
[root@ip-172-31-35-122 scripts]# vi monitor-cpu.bash
[root@ip-172-31-35-122 scripts]# cat monitor-cpu.bash
#!/bin/bash

echo 'Monitoring the Cpu Utlization....'
echo
cpu_num=`top -bn1 | grep 'Cpu(s)' | awk -F ',' '{print $4}' | awk '{print 100-$1}'`
cpu=${cpu_num%.*}

if [ $cpu -gt 80 ]; then
        echo "Cpu utilization is currently at $cpu" | mail -s 'Cpu utilization Report' mohdfaisal1477@gmail.com
        echo 'Email has been sent'
else
        echo "Cpu utilization is currently at $cpu"
fi
```

```
[root@ip-172-31-35-122 scripts]# bash ./monitor-cpu.bash
Monitoring the Cpu Utlization....

Cpu utilization is currently at 6
```

11. Create a bash script to monitor disk space and if it is ,more than 80% then send email notification.

Step 1: -Create a bash file with Touch or vi command

Step 2: -Use shebang Character and shell path to be used for kernel

Step 3:- First we get the disk usage with df / for entire root path.

Step 4:- Then get second row fifth column using AWK. Finally use SED to remove percentage symbol.

Step 5:- Store this in variable and then check if is greater than 80 or not

Step 6:-If greater then use mail command to send a report

Step 7:- If not then print a normal message with the utilization value.

Step 8:- Finally run this file with help of bash command

```
[root@ip-172-31-35-122 scripts]# vi monitor-disk.bash
[root@ip-172-31-35-122 scripts]# cat monitor-disk.bash
#!/bin/bash

disk=$(df / | awk 'NR==2 {print $5}' | sed 's/%//')

echo 'Monitoring the disk usage....'

if [ $disk -gt 80 ]; then
    echo "Warning: Disk usage is above 80%!" | mail -s 'Disk Usage Report' mohdfaisal1477@gmail.com
else
    echo "Disk usage is normal."
    echo "Disk Usage: $disk%"
fi
```

```
[root@ip-172-31-35-122 scripts]# bash ./monitor-disk.bash
Monitoring the disk usage....
Disk usage is normal.
Disk Usage: 26%
[root@ip-172-31-35-122 scripts]#
```

12. Create a bash script to monitor memory and if it is ,more than 80% then send email notification.

Step 1: -Create a bash file with Touch or vi command

Step 2: -Use shebang Character and shell path to be used for kernel

Step 3:- First we get the memory usage with free command.

Step 4:- Then get filter the line using AWK. Also use “%.0f” to round off the value and calculate the percentage with used / total * 100.

Step 5:- Store this in variable and then check if is greater than 80 or not

Step 6:-If greater than 80 use mail command to send a report

Step 7:- If not then print a normal message with the utilization value.

Step 8:- Finally run this file with help of bash command

```
[root@ip-172-31-35-122 scripts]# vi monitor-mem.bash
[root@ip-172-31-35-122 scripts]# cat monitor-mem.bash
#!/bin/bash

mem=`free | awk '/Mem:/ {printf "%.0f", ($3/$2)*100}'`

echo "Monitoring The memory usage...."

if [ $mem -gt 80 ]; then
    free -h | mail -s "Memory Usage Report" mohdfaisal@gmail.com
    echo "Memory usage exceeded 80%. Report email attempted."
else
    echo "Memory usage is normal."
    echo "Memory utilization: $mem%"
fi
```

```
[root@ip-172-31-35-122 scripts]# bash monitor-mem.bash
Monitoring The memory usage....
Memory usage is normal.
Memory utilization: 18%
[root@ip-172-31-35-122 scripts]#
```

Exercise

Crontab

1. April 5th Midnight

Step 1:- To create an entry use crontab -e

Step 2 :- Add time 0 0 5 4 * and file path then save

Step 3:- To view the list crontab -l

```
[ec2-user@ip-172-31-35-122 ~]$ crontab -e
no crontab for ec2-user - using an empty one
crontab: installing new crontab
[ec2-user@ip-172-31-35-122 ~]$ crontab -l
0 0 5 4 * /home/ec2-user/text.bash
```

2. 5th of Every November, January, June if it is a Thursday

Step 1:- To create an entry use crontab -e

Step 2 :- Add time * * 5 1,6,11 4 and file path then save

Step 3:- To view the list crontab -l

```
[ec2-user@ip-172-31-35-122 ~]$ crontab -e
crontab: installing new crontab
[ec2-user@ip-172-31-35-122 ~]$ crontab -l
0 0 5 4 * /home/ec2-user/text.bash
* * 5 1,6,11 4 /home/ec2-user/text.bash
[ec2-user@ip-172-31-35-122 ~]$
```

3. At 05 and 27th minutes of 9, 10, 11 hours every day

Step 1:- To create an entry use crontab -e

Step 2 :- Add time 5,27 9,10,11 * * * and file path then save

Step 3:- To view the list crontab -l

```
[ec2-user@ip-172-31-35-122 ~]$ crontab -e
crontab: installing new crontab
[ec2-user@ip-172-31-35-122 ~]$ crontab -l
0 0 5 4 * /home/ec2-user/text.bash
* * 5 1,6,11 4 /home/ec2-user/text.bash
5,27 9,10,11 * * * /home/ec2-user/text.bash
```

4. 34 minutes of 9th hour on 15th August

Step 1:- To create an entry use crontab -e

Step 2 :- Add time 34 9 15 8 * and file path then save

Step 3:- To view the list crontab -l

```
[ec2-user@ip-172-31-35-122 ~]$ crontab -e
crontab: installing new crontab
[ec2-user@ip-172-31-35-122 ~]$ crontab -l
0 0 5 4 * /home/ec2-user/text.bash
* * 5 1,6,11 4 /home/ec2-user/text.bash
5,27 9,10,11 * * * /home/ec2-user/text.bash
34 9 15 8 * /homr/ec2-user/text.bash
```

5. Every midnight

Step 1:- To create an entry use crontab -e

Step 2 :- Add time 0 0 * * * and file path then save

Step 3:- To view the list crontab -l

```
[ec2-user@ip-172-31-35-122 ~]$ crontab -e
crontab: installing new crontab
[ec2-user@ip-172-31-35-122 ~]$ crontab -l
0 0 5 4 * /home/ec2-user/text.bash
* * 5 1,6,11 4 /home/ec2-user/text.bash
5,27 9,10,11 * * * /home/ec2-user/text.bash
34 9 15 8 * /homr/ec2-user/text.bash
0 0 * * * /home/ec2-user/text.bash
[ec2-user@ip-172-31-35-122 ~]$
```

6. Every Weekend (Saturday night 11:59 PM)

Step 1:- To create an entry use crontab -e

Step 2 :- Add time 59 23 * * 6 and file path then save

Step 3:- To view the list crontab -l

```
[ec2-user@ip-172-31-35-122 ~]$ crontab -e
crontab: installing new crontab
[ec2-user@ip-172-31-35-122 ~]$ crontab -l
0 0 5 4 * /home/ec2-user/text.bash
* * 5 1,6,11 4 /home/ec2-user/text.bash
5,27 9,10,11 * * * /home/ec2-user/text.bash
34 9 15 8 * /homr/ec2-user/text.bash
0 0 * * * /home/ec2-user/text.bash
59 23 * * 6 /home/ec2-user/text.bash
[ec2-user@ip-172-31-35-122 ~]$
```

7. After every reboot

Step 1:- To create an entry use crontab -e

Step 2 :- Add @reboot for every reboot and file path then save

Step 3:- To view the list crontab -l

```
[ec2-user@ip-172-31-35-122 ~]$ crontab -e
crontab: installing new crontab
[ec2-user@ip-172-31-35-122 ~]$ crontab -l
0 0 5 4 * /home/ec2-user/text.bash
* * 5 1,6,11 4 /home/ec2-user/text.bash
5,27 9,10,11 * * * /home/ec2-user/text.bash
34 9 15 8 * /homr/ec2-user/text.bash
0 0 * * * /home/ec2-user/text.bash
59 23 * * 6 /home/ec2-user/text.bash
@reboot /home/ec2-user/text.bash
```